

ATIVIDADE: PROTOTIPAGEM COM ARDUINO

ALUNO(A) : Calyme Vilhena dos Santos

PROFESSOR : Emmerson Santa Rita da Siva

CURSO: TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

INTRODUÇÃO

Este relatório apresenta o desenvolvimento de uma atividade prática utilizando a plataforma Arduino, com foco no controle de LEDs por meio de botões e na leitura de sinais digitais. A proposta da atividade foi compreender o funcionamento das entradas e saídas do microcontrolador, bem como a interação entre componentes eletrônicos em um circuito prototipado.

Durante a prática, foram explorados conceitos fundamentais como leitura de botões, acionamento de LEDs, uso do modo INPUT_PULLUP e controle de contagem de cliques por meio da comunicação serial. Também foram aplicadas técnicas para melhorar o funcionamento do sistema, como o uso de debounce e o controle de tempo com millis(), visando tornar o programa mais eficiente e estável.

Dessa forma, a atividade contribui para o entendimento dos princípios básicos de sistemas embarcados e da programação aplicada ao Arduino, permitindo a construção de soluções simples de automação e controle eletrônico.

QUESTÃO 1

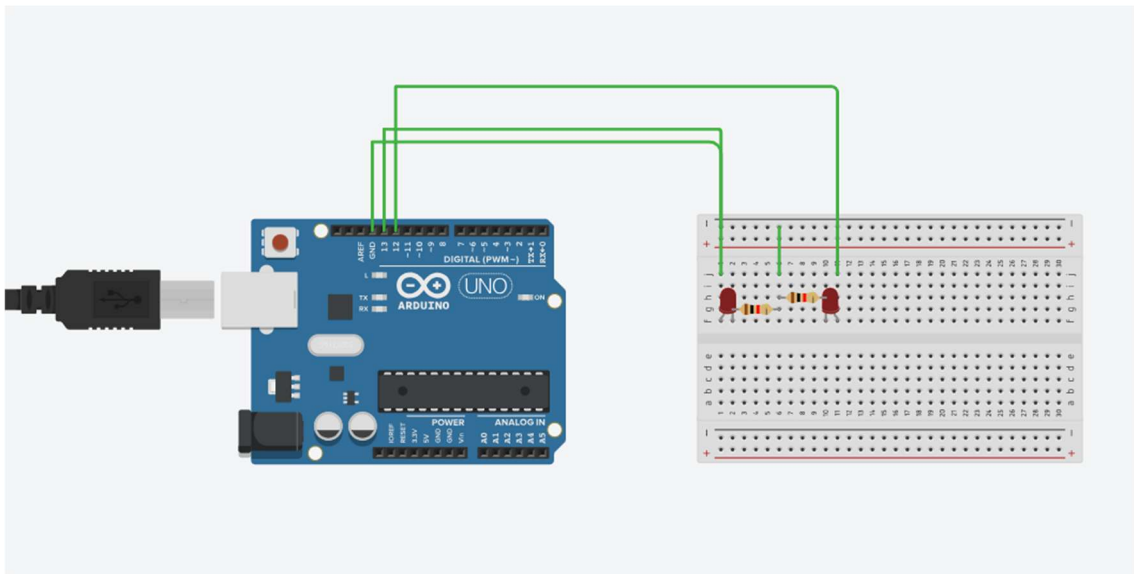
Adicione um segundo LED que acende quando o primeiro apaga (lógica invertida).

CÓDIGO:

```
void setup()
{
  pinMode(13, OUTPUT);
  pinMode(12, OUTPUT);
}

void loop()
{
  digitalWrite(13, 1);
  digitalWrite(12, 0);
  delay(400);
  digitalWrite(13, 0);
  digitalWrite(12, 1);
  delay(400);
}
```

PROTOTIPAGEM:



QUESTÃO 2

Use dois botões — um para acender, outro para apagar (sem toggle).

Nessa questão trabalhamos com o uso dos botões algo parecido com interruptores de luz, um liga e o outro desliga, os botões estão conectados ao polo negativo através de um resistor cada botão, eles não são conectados ao polo positivo pois o arduino já fornece conexão positiva internamente quando se é usado o INPUT_PULLUP, e o que seria essa função? Bom ela serve para inicializar o nosso botão com o valor HIGH e inverte o valo quando chega nas condições, por isso (digitalRead(botao1) == LOW.

CÓDIGO:

```
int led1 = 13;
int led2 = 12;
int botao1 = 3;
int botao2 = 2;

void setup()
{
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  pinMode(botao1, INPUT_PULLUP);
  pinMode(botao2, INPUT_PULLUP);
}

void loop()
{
  if(digitalRead(botao1) == LOW){

    digitalWrite(led1, HIGH);
    digitalWrite(led2, HIGH);

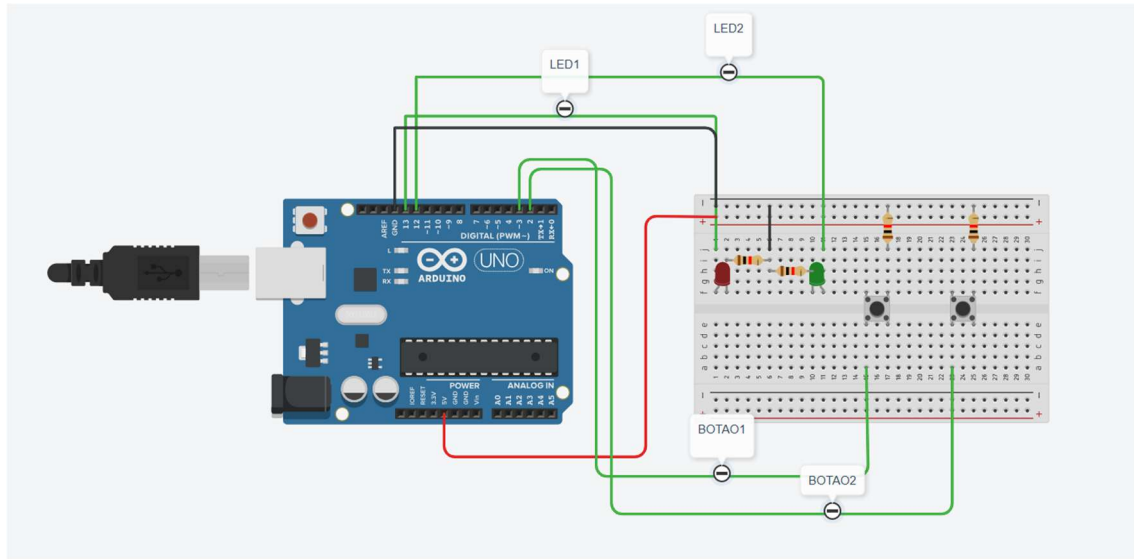
  }

  if(digitalRead(botao2) == LOW){

    digitalWrite(led1, LOW);
    digitalWrite(led2, LOW);

  }
}
```

PROTOTIPAGEM:



QUESTÃO 3

Implemente um contador no Serial Monitor que exibe quantas vezes o botão foi pressionado. Use `Serial.begin(9600)` e `Serial.println(contador)`.

O serial monitor serve para fazer uma conexão entre dispositivos e no caso do Arduino ele já está integrado fisicamente no hardware com os pinos de transmissão e recepção, chamados pinos TX e RX, eles são conectados ao computador a partir do momento em que é conectado o USB no computador, não dependendo de ligações com jumpers ou algo do tipo.

O `Serial.begin(9600)` define a velocidade em que será processado, nesse caso 9600 bits por segundo.

O `Serial.println(cont)` vai nos mostrar no Serial monitor a quantidade de cliques no botão

CÓDIGO:

```
int led1 = 13;
int led2 = 12;
int botao1 = 3;
int botao2 = 2;

int cont = 0;

void setup()
{
  //inicializa o serial com a velocidade em bits/s
  Serial.begin(9600);
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  pinMode(botao1, INPUT_PULLUP);
  pinMode(botao2, INPUT_PULLUP);
}

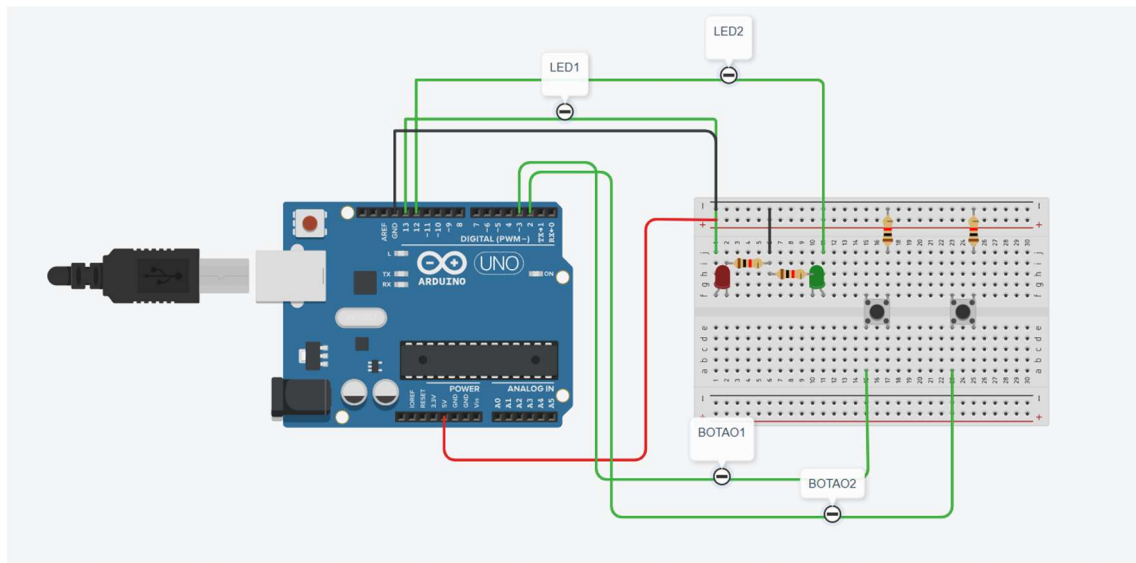
void loop()
{
  if(digitalRead(botao1)== LOW){

    digitalWrite(led1, HIGH);
    digitalWrite(led2, HIGH);
    cont++;
    delay(400);
  }

  if(digitalRead(botao2) == LOW){

    digitalWrite(led1, LOW);
    digitalWrite(led2, LOW);
    cont++;
    delay(400);
  }
  Serial.println(cont);
}
```

PROTOTIPAGEM:



QUESTÃO 4

Elimine o `delay()` de `debounce` e implemente um `debounce` por tempo usando `millis()` — isso é importante para não travar o programa com `delay()`.

A questão 4 é uma atualização da questão 3 onde foram introduzidas as funções `millis()` e `debounce`. A função `millis()` serve para ler a quantidade de tempo em que o programa está sendo executado independente se todas as condições estão sendo executadas ou não. Já o `debounce` serve como um tratamento de sinal digital, onde ele aguarda milissegundos e faz a validação do sinal mais estável, pois quando um botão é clicado são geradas várias frequências o `debounce` serve para identificar e manter somente 1, evitando assim erros no processamento do clique.

Na condição **`millis() - tempoAnterior > debounce`**, ele verifica se o valor do `millis()` já é maior que o do `debounce` que seria 400, caso seja verdadeiro ele deixa executar o próximo clique.

CÓDIGO:

```
int led1 = 13;  
int led2 = 12;  
int botao1 = 3;  
int botao2 = 2;
```

```
int cont = 0;
```

```
long tempoAnterior = 0;
```

```
//o botao é apertado aguarda alguns milisegundos verifica o sinal mais estavel e  
torna ele o unico verdadeiro  
long debounce = 400;
```

```
void setup()  
{  
  Serial.begin(9600);  
  pinMode(led1, OUTPUT);  
  pinMode(led2, OUTPUT);  
  pinMode(botao1, INPUT_PULLUP);  
  pinMode(botao2, INPUT_PULLUP);  
}  
  
void loop()  
{  
  if(millis() - tempoAnterior > debounce)  
  {  
    if(digitalRead(botao1) == LOW){  
  
      digitalWrite(led1, HIGH);  
      digitalWrite(led2, HIGH);  
      cont++;  
      tempoAnterior = millis();  
    }  
  
    if(digitalRead(botao2) == LOW){  
  
      digitalWrite(led1, LOW);  
      digitalWrite(led2, LOW);  
      cont++;  
      tempoAnterior = millis();  
    }  
  }  
  Serial.println(cont);  
}
```

PROTOTIPAGEM:

